UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Laboratorio No. 11

Data Science

Sección: 10

Jose Ordoñez - 231329

Gadiel Ocaña - 231270

Adrián Gonzáles - 23152

José Antón - 221041

Guatemala, 2026

Defina el alcance del flujo

Variable, unidad y nivel de agregación

Tabla 1. Variable, unidad y agregación

Elemento	Definición
Variable estimada	Llegadas internacionales de viajeros a Guatemala por mes.
Unidad de medida	Viajeros por mes (personas/mes). No son personas únicas, pernoctaciones, gasto turístico ni visitantes por departamento.
Nivel de agregación	Mensual. Total nacional y seis segmentos: Total, Vía Aérea, Vía Terrestre, Vía Marítima, América del Centro, América del Norte y Europa.
Cobertura observada	Enero 2009 – junio 2026 (210 meses). 2026 solo contiene enero–junio.
Universo	Registros de INGUAT/Migración procesados en los Laboratorios 1 y 2. Las vistas por país o frontera son descriptivas y no se publican como estimaciones modeladas.

Fuentes y columnas

Tabla 2. Fuentes y columnas utilizadas

Elemento	Definición
Fuente primaria	Base_Migracion_2009-2026jun.xlsx (INGUAT/Migración), corte a junio 2026. El archivo no se incorpora a la imagen Docker; se monta como dato institucional.
Columnas utilizadas	Año, Mes cod, Vía, Frontera, País, Región dos, Tipo de Viajero y Viajero.
Archivos derivados	data/base_viajeros.csv.gz (detalle), data/series_mensuales.csv (serie observada), data/predicciones_lab1.csv y data/metricas_lab1.csv (resultados del Laboratorio 1).
Construcción de la serie	Suma de Viajero por Año y Mes cod. No se construye a partir de presencias diarias ni de una fuente distinta a la mensual.
Filtro metodológico	Se excluye Tipo de Viajero == "Viajero" en todas las series, excepto en Vía Marítima, que se usa completa. Esto responde al cambio de metodología 2022–2023 y evita duplicar visitantes.
Faltantes y fechas	Un mes sin fila en la fuente es un faltante, no un cero. Un cero solo es válido si la fuente lo reporta. Las fechas inválidas o los períodos esperados sin cobertura detienen el flujo.

Factor de reparto

El factor de reparto se documenta como el conjunto de proporciones con que la fuente distribuye el total de viajeros entre Turista, Excursionista, Viajero y Cruceristas. En la base recibida ese reparto ya está aplicado: por eso la columna Viajero contiene decimales y la suma por grupo recupera el total. El pipeline no vuelve a aplicar el factor, conserva los decimales hasta la

agregación y redondea solo para presentación. Si el equipo dispone de la tabla explícita de factores, se guarda en config/factores_reparto.csv y se versiona. Para la asignación a destinos se usa el factor oficial de la guía; en esta versión nacional cada llegada se cuenta una vez, por lo que el factor de asignación es 1.0.

Períodos de ajuste, evaluación y publicación

Tabla 3. Períodos del flujo

Etapa	Período	Uso
Ajuste base	2009-01 a 2021-03 (147 meses)	Entrenamiento con el mismo corte del Laboratorio 1.
Evaluación histórica	2021-04 a 2026-06 (63 meses)	Comparación con los resultados del Laboratorio 1. No es la única base de aceptación porque es un pronóstico multimensual y contiene la recuperación pospandemia.
Backtest de aceptación	2025-07 a 2026-06 (12 cortes)	Evaluación principal con origen expansivo y un paso hacia adelante: en cada corte se ajusta solo con información anterior al mes evaluado.
Pronóstico publicado	2026-07 a 2026-12 (6 meses)	Estimaciones hacia adelante con el corte de junio 2026.
Sensibilidad ene–feb 2026	Enero y febrero 2026	Se pueden ocultar y estimar, pero se documenta la limitación: el corte actual ya contiene esos valores y el código se especificó después de verlos; además, la pandemia y el cambio metodológico impiden un contrafactual limpio. Si no hay observaciones comparables, no se calculan errores.

Parámetros y supuestos

Tabla 4. Parámetros y supuestos del procedimiento

Parámetro o supuesto	Valor o criterio
ARIMA por serie	Total (1,0,1); Aérea (1,0,1); Terrestre (2,1,0); Marítima (2,0,3); Centro (1,0,0); Norte (3,0,0); Europa (4,1,2).
Seasonal Naive	Repite el mismo mes del año anterior. Se usa como referencia sencilla.
Prophet	Estacionalidad anual activa; semanal y diaria desactivadas.
Transformación logarítmica	Se aplica solo si la diferencia entre el CV anual máximo y mínimo es mayor que 0.25 y todos los valores son positivos. La regla queda en configuración y se registra por serie.

Semilla y dependencias	Semilla 20261029 para cualquier componente aleatorio. Versiones fijadas en requirements.txt o en el archivo de bloqueo y registradas en MLflow.
Selección de modelo	Los parámetros se fijan con el Laboratorio 1 y no se reajustan con el período de prueba. La regla de aceptación se define antes de ver los resultados y no se modifica después.
Aditividad	Las filas de la fuente son aditivas dentro de mes, vía, frontera, país, región y tipo de viajero. Si hay duplicados, se documenta si son registros parciales aditivos o un error; no se eliminan silenciosamente.
Continuidad	Las estimaciones suponen que el patrón histórico y la estacionalidad se mantienen. No anticipan pandemias, cambios metodológicos ni shocks externos.
Revisiones de la fuente	Si INGUAT revisa datos históricos, la versión publicada puede cambiar; por eso se registra el hash de la fuente y la fecha de generación.
MAPE y redondeo	El MAPE se calcula solo cuando las observaciones son mayores que cero. Si hay ceros, se reporta cuántas observaciones se excluyeron y se usan MAE/RMSE como métricas principales. Los valores se conservan con decimales en el archivo; el tablero redondea solo para mostrar.

Archivo de resultados que utilizará el tablero

La publicación se organiza en versiones inmutables y un puntero atómico. El tablero lee únicamente current.json y el archivo de la versión aprobada; nunca lee un archivo candidato. Si el puntero no existe o está corrupto, muestra “estimaciones no disponibles”.

Tabla 5. Esquema mínimo de estimaciones.csv

Columna	Descripción
destino	Guatemala (o el destino asignado, si aplica).
serie	Total, Vía Aérea, Vía Terrestre, Vía Marítima, América del Centro, América del Norte o Europa.
periodo	Mes estimado, formato YYYY-MM-01.
valor	Viajeros por mes.
unidad	viajeros/mes.
modelo	Modelo que generó la estimación.
conjunto	evaluacion o pronostico.
horizonte	Meses hacia adelante; 0 en backtest.
run_id	Identificador de la ejecución en MLflow.
fecha_generacion	Fecha y hora ISO 8601.
version_procedimiento	Versión publicada, por ejemplo v2026.10.29-01.

fuente_hash / config_hash	SHA-256 de la fuente y de la configuración.
estado	aprobado o referencia. El tablero usa aprobado por defecto.

metricas.csv incluye serie, modelo, periodo_inicio, periodo_fin, n_obs, MAE, RMSE, MAPE_pct, baseline, baseline_MAE, baseline_RMSE, cumple_regla y run_id. controles.csv incluye control, severidad, estado, n_afectados, detalle, run_id y timestamp. manifest.json conserva la versión, el run_id, el commit de Git, los hashes, los períodos, la fecha de aprobación y el resumen de métricas.

Diagrama del flujo y puntos de control

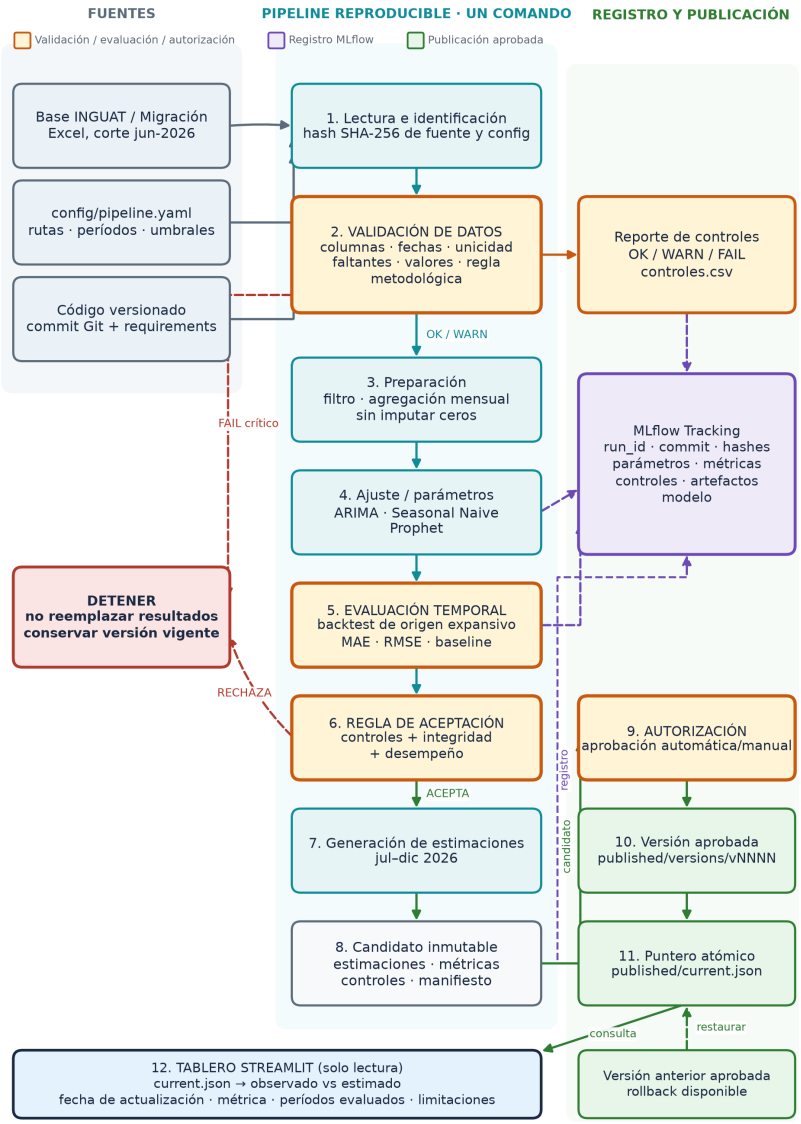


Figura 1. Flujo MLOps del Observatorio de Turismo Sostenible. Elaboración propia.

La validación ocurre antes de preparar las series: si falla un control crítico, el pipeline se detiene y no reemplaza los resultados vigentes. La evaluación se realiza después del ajuste y antes de generar la publicación; usa un backtest de origen expansivo para que cada estimación se calcule solo con información anterior al mes evaluado. La autorización ocurre después de registrar el candidato en MLflow; solo si se cumplen los controles y la regla de aceptación se actualiza el puntero de la versión aprobada.

Tabla 6. Puntos de validación, evaluación y autorización

Punto de control	Etapa	Qué se decide	Evidencia
Validación de datos	Etapa 2	Si los datos son aptos. Un FAIL crítico detiene el pipeline antes de preparar o reemplazar resultados.	Reporte de controles y artefacto en MLflow.
Evaluación del procedimiento	Etapa 5	Si el modelo es aceptable. Se usan solo datos anteriores a cada corte; no hay fuga de información futura.	MAE, RMSE, MAPE y comparación con la referencia.
Autorización de publicación	Etapa 9	Si el candidato reemplaza la versión vigente. Requiere controles críticos en OK, salidas válidas y cumplimiento de la regla de aceptación.	Run de MLflow y registro de aprobación.
Publicación	Etapas 10–11	Se guarda una versión inmutable y se actualiza el puntero de forma atómica. Si falla, se conserva la versión anterior.	current.json y manifest.json.
Consulta	Etapa 12	El tablero lee únicamente la versión aprobada y muestra fecha de actualización, métrica de validación, períodos evaluados y limitaciones.	Captura del tablero y versión consultada.

La regla de aceptación queda parametrizada en config/pipeline.yaml. Su estructura es: (i) todos los controles críticos en OK; (ii) todas las estimaciones finitas, no negativas y dentro de un rango plausible; (iii) MAE/RMSE del candidato por debajo de umbrales absolutos según la escala de la serie y no peor que la referencia en más de un margen justificado; (iv) si no hay observaciones comparables, no se calculan errores ni se aprueba por desempeño. Los valores numéricos exactos se fijan en la sección 5 y se congelan antes de ejecutar.

Laboratorio 11 — MLOps aplicado al Observatorio de Turismo Sostenible

Secciones 5 a 8: regla de aceptación, tablero publicado, monitoreo y Docker. Este bloque continúa el informe de las secciones 1–4 que se conserva en las páginas anteriores.

5. Regla de aceptación congelada y versiones aprobadas

5.1 Decisión congelada

La regla de aceptación se definió y congeló el **2026-11-01**, antes de la corrida oficial, y no se modificó después de ver los resultados. Vive en `config/pipeline.yaml` (`aceptacion:`), se copia al `manifest.json` de cada versión y se registra en MLflow.

Esta sección reemplaza el carácter provisional que tenía la regla en el informe de las secciones 1–4. Las corridas del 31 de octubre quedan como evidencia histórica de intentos rechazados; no se usaron para ajustar la regla final.

5.2 Regla aplicada

Criterio	Valor congelado	Efecto
Error relativo	`MAE <= 20 %` del nivel medio observado en el backtest	Si una serie no cumple, se publica como referencia con Seasonal Naive
Observaciones mínimas	6 observaciones comparables	Menos de 6: no se evalúa por desempeño
Serie obligatoria	`Total`	Si `Total` no cumple, la versión se rechaza
Controles críticos	C1–C6 en OK (los WARN se documentan)	Un FAIL crítico detiene el pipeline antes de publicar
Salidas	Finitas, no negativas y dentro del rango plausible	Si falla, el candidato se rechaza
Referencia	Seasonal Naive se calcula y se reporta	**No es una compuerta de aceptación** (laboratorio, 5.5)

El umbral de 20 % es una tolerancia de planificación, no una promesa de precisión puntual. La referencia se conserva porque ayuda a interpretar el error, pero el laboratorio no exige superarla; cuando ARIMA no la supera, se documenta sin bloquear la publicación.

5.3 Resultado del backtest

El backtest usa origen expansivo y un paso hacia adelante, con 12 cortes mensuales de **julio 2025 a junio 2026**. En cada corte el modelo se ajusta solo con información anterior al mes evaluado.

Serie	MAE (viajeros/mes)	MAE / nivel	MAE / referencia	Estado publicado
Total	40,387	14.7 %	3.36×	aprobado (ARIMA)
Vía Aérea	14,714	14.1 %	1.57×	aprobado (ARIMA)
Vía Terrestre	25,376	14.8 %	2.03×	aprobado (ARIMA)
Vía Marítima	554	93.7 %	8.27×	referencia (Seasonal Naive)
América del Centro	24,862	14.6 %	2.04×	aprobado (ARIMA)
América del Norte	14,254	19.6 %	2.21×	aprobado (ARIMA)
Europa	3,963	26.0 %	3.44×	referencia (Seasonal Naive)

Cinco series cumplen y se publican con ARIMA. Vía Marítima y Europa no cumplen el 20 %: sus valores se publican como **referencia** con Seasonal Naive y el tablero lo indica. No se presenta como aprobado un modelo que no cumple la regla.

5.4 Versiones y reproducibilidad

Se generaron las versiones locales `v2026.11.01-01` a `v2026.11.01-04`. Las versiones `-03` y `-04` incluyen monitoreo y son equivalentes: `scripts/consultar_mlflow.py comparar` reportó **126 estimaciones y 79 métricas comparadas con diferencia máxima** `0.000e+00`. ARIMA no tiene componentes aleatorios; la reproducibilidad se evalúa en el mismo entorno.

El puntero final de la entrega local apunta a `v2026.11.01-04`. La corrida de Docker generó además `v2026.11.02-01`, que se conserva como evidencia de la ejecución en contenedor. Sus decisiones son las mismas; Europa cambia de 3,963 a 3,825 de MAE porque la imagen usa otra versión de Python y el optimizador de ARIMA se detiene en un punto distinto cuando no converge. La corrida local es la canónica.

6. Tablero integrado con la versión publicada

6.1 Contrato de lectura

El tablero no reajusta modelos ni lee candidatos. `app/publicacion.py` lee `published/current.json` y, desde la ruta indicada, carga `manifest.json`, `metricas.csv`, `estimaciones.csv`, `backtest_detalle.csv`, `reporte_validacion.md` y `monitoreo.json` cuando existen. El puntero es el único contrato entre el pipeline y el tablero.

6.2 Qué muestra la pestaña ⑤

- Versión aprobada, fecha de actualización, `run_id` y `mlflow_run_id`.
- Hashes de fuente y configuración, destino y unidad.
- Tabla de validación con `MAE`, `MAE / nivel`, `MAE / referencia`, meses evaluados y estado por serie.
- Distinción visual entre **observado** (verde azulado) y **estimación** (naranja si está aprobada; gris discontinuo si es referencia).
- Alertas de monitoreo de la última ejecución, con la aclaración de que no demuestran por sí solas pérdida de precisión.
- Limitaciones: jul–dic 2026 son estimaciones, el backtest es corto, las series de referencia no pasaron la regla y la pandemia impide un contrafactual limpio.

La pestaña ④ también muestra la estimación publicada para julio–diciembre de 2026 y etiqueta si la serie es aprobada o de referencia.

6.3 Estimaciones no disponibles

Si `current.json` no existe, está corrupto, la versión está incompleta o no está aprobada, el cargador devuelve `None` con un motivo y la interfaz muestra **«Estimaciones no disponibles»**. No se inventan valores ni se reajusta un modelo. La última versión válida que exista en disco no se modifica; el pipeline solo mueve el puntero cuando publica una versión aprobada.

6.4 Pruebas

`tests/test_tablero_publicacion.py` cubre: ausencia de versión, versión incompleta o no aprobada, lectura de una versión aprobada con estados por serie, restauración que cambia el puntero y `current.json` corrupto. La suite completa tiene **41 pruebas** con datos sintéticos y no requiere el dato institucional.

7. Monitoreo, fallo controlado y recuperación

7.1 Tres verificaciones

Cada ejecución escribe `monitoreo.csv`, `monitoreo.json` y `reporte_monitoreo.md` en el candidato, en la versión publicada y como artefactos de MLflow.

Verificación	Qué comprueba	Alerta cuando
M1 actualización	La fuente llega al último mes de cobertura y el pronóstico cubre el período configurado	desfase mayor a 0 meses o pronóstico incompleto
M2 calidad	Faltantes, duplicados y meses sin información frente a la línea base de la versión anterior	faltantes +0.5 pp, duplicados +0.1 pp o cobertura +0 meses
M3 resultados	Negativos, valores no finitos y variación mes a mes o frente a la versión anterior	variación mensual > 35 % o frente a versión > 25 %

Las alertas **no bloquean** la publicación por sí solas: son una señal para revisión. El error predictivo solo se actualiza con observaciones comparables; no se inventan errores para meses futuros. La línea base de M2 y las estimaciones previas quedan en `monitoreo.json` para la siguiente ejecución.

7.2 Resultado de la corrida canónica

M1 y M2 quedaron en **OK**. M3 generó una **ALERTA** real por la variación mes a mes de Europa (94.4 %) y Vía Marítima (36 %). Son series publicadas como referencia, con estacionalidad irregular; la alerta se registra y se muestra en el tablero, pero no cambia la versión publicada. Una variación alta es una señal para revisión, no una prueba de pérdida de precisión.

7.3 Prueba controlada de fallo

Se ejecutó el pipeline con una copia de la fuente a la que se le eliminó la columna `País`.

Evidencia	Resultado
Run local	`20261101-195740-3246f1`
Estado MLflow	`FAILED`
Código de salida	`2`
Etapa final	`2_validacion`
Causa	`C1 columnas y tipos: Faltan columnas indispensables: ['País']`
Métricas de error	No se calcularon
`published/current.json`	SHA-256 sin cambios (`d855739c…`)

El flujo se detuvo antes de preparar o reemplazar resultados. La versión vigente siguió siendo la misma y el tablero no perdió su fuente de datos.

7.4 Recuperación y restauración

Al corregir la fuente, la siguiente corrida publicó `v2026.11.01-03` y luego `v2026.11.01-04`; ambas son equivalentes (diferencia máxima `0.000e+00`). También se demostró la restauración de una versión anterior:

```
python -m pipeline.run --restaurar v2026.11.01-02
python -m pipeline.run --restaurar v2026.11.01-04
```

El tablero pasó a leer `v2026.11.01-02` y después volvió a `v2026.11.01-04`. Restaurar solo cambia el puntero; las versiones publicadas no se reescriben. El `current.json` restaurado registra `restaurada_en` y `version_anterior`.

8. Contenedores, persistencia y copia limpia

8.1 Servicios

El `Dockerfile` usa `python:3.12-slim` y una sola imagen para tablero, pipeline y MLflow. `docker-compose.yml` define:

Servicio	Comando	Puerto host	Volúmenes
`mlflow`	`mlflow server` con SQLite y artefactos	`5001 → 5000`	`./mlflow:/app/mlflow`
`tablero`	`streamlit run app/app.py`	`8501 → 8501`	`./data` y `./published` en solo lectura
`pipeline`	`python -m pipeline.run`	—	`./data/raw` en solo lectura, `./published` y `./runs`

El pipeline usa `MLFLOW_TRACKING_URI=http://mlflow:5000` y espera a que MLflow esté `healthy`. El puerto 5000 del host suele estar ocupado en macOS; por eso el tablero de MLflow se publica en 5001, pero dentro de la red el servicio sigue en `mlflow:5000`.

8.2 Aislamiento del dato institucional

`.dockerignore` excluye `data/raw/*`, `runs/`, `mlflow/`, `mlruns/` y `.git/`. El `.xlsx` no se copia a la imagen: se monta en solo lectura únicamente en el servicio `pipeline`. La verificación dentro de la imagen mostró que `/app/data/raw/` solo contiene `.gitkeep` y que `find /app -name "*.xlsx"` no devuelve archivos.

8.3 Persistencia

Se ejecutó `docker compose down`, que eliminó contenedores y red. Después:

- `published/current.json` conservó el mismo SHA-256 (`c78560c4…`).
- `mlflow/mlflow_docker.db` siguió en disco.
- El run `48daaef8440d…` siguió en la base con estado `FINISHED`.
- Al recrear con `docker compose up -d mlflow tablero`, el run siguió disponible por la API y el tablero respondió HTTP 200.

La corrida de Docker publicó `v2026.11.02-01` con la misma regla congelada y las mismas decisiones de aceptación.

8.4 Prueba desde una copia limpia

Se clonó el repositorio en un directorio temporal, se copió solo el `.xlsx` institucional a `data/raw/` y se ejecutó Compose desde el clon. La corrida terminó con código `0` y publicó `v2026.11.02-02` (`run_id 20261102-021301-d6d6d5`, MLflow `ed692d72e66a…`). `mlflow/mlflow_docker.db` se creó desde cero y el run quedó `FINISHED`. El clon solo mostró los cambios esperados: `published/current.json` modificado y la versión nueva sin seguimiento. No se usaron `runs/`, `mlflow/` ni archivos locales sin versionar de la copia original.

8.5 Nota de alcance

La base local `mlflow/mlflow.db` conserva una ruta de artefactos específica del host. Para no romper las corridas locales, Docker usa `mlflow/mlflow_docker.db` y artefactos en `/app/mlflow/artifacts`; ambas bases viven bajo `mlflow/`, que no se versiona. La imagen no incluye `.git`, por lo que el manifiesto de la corrida Docker registra `commit_git: sin-git`; la trazabilidad se completa con la versión publicada, los hashes de fuente y configuración, las dependencias y el registro en MLflow. La corrida local canónica sí registra el commit de Git.

9. Cierre y evidencia de la entrega

Requisito	Evidencia
Regla de aceptación congelada	`config/pipeline.yaml`, `manifest.json` y MLflow; fecha 2026-11-01
Versiones aprobadas	`v2026.11.01-01` a `v2026.11.01-04`; puntero final en `-04`

Requisito	Evidencia
Tablero integrado	Pestaña ⑤ y pronóstico publicado; lectura de `current.json`; pruebas de versión ausente, incompleta y restauración
Monitoreo	M1–M3 en `monitoreo.csv/json` y `reporte_monitoreo.md`; alertas no bloqueantes
Fallo y recuperación	Run `20261101-195740-3246f1` con código 2; recuperación a `-03`/`-04`; restauración a `-02` y vuelta a `-04`
Docker	`Dockerfile`, `docker-compose.yml`, persistencia tras `down`, sin `.xlsx` en la imagen, copia limpia con `v2026.11.02-02`
Registro de ejecuciones	`docs/mlflow_ejecuciones.csv` con 3 corridas históricas, `v01`–`v04` y la corrida fallida; bases y artefactos en `mlflow/`, entregados aparte

El flujo cumple el objetivo del laboratorio: actualizar estimaciones de forma controlada, identificar qué datos, código y configuración produjeron cada resultado y no reemplazar la versión vigente cuando un control o la regla de aceptación falla.